

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
“ЛЭТИ” ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»
КАФЕДРА МОЭВМ**

**ОТЧЕТ
по лабораторно-практической работе № 5
«Сохранение и загрузка данных из файла»
по дисциплине «Объектно - ориентированное программирование на
языке Java»**

Выполнил: Шарапов И.Д.

Факультет: КТИ

Группа: №3312

Подпись преподавателя: _____

Санкт-Петербург

2024

Содержание

Цель работы	3
Пример работы программы	3
Содержимое файлов	9
Текст программы	10
Приложение	17

Цель работы

Знакомство с организацией обмена данными между объектами экранной формы и файлом.

Пример работы программы

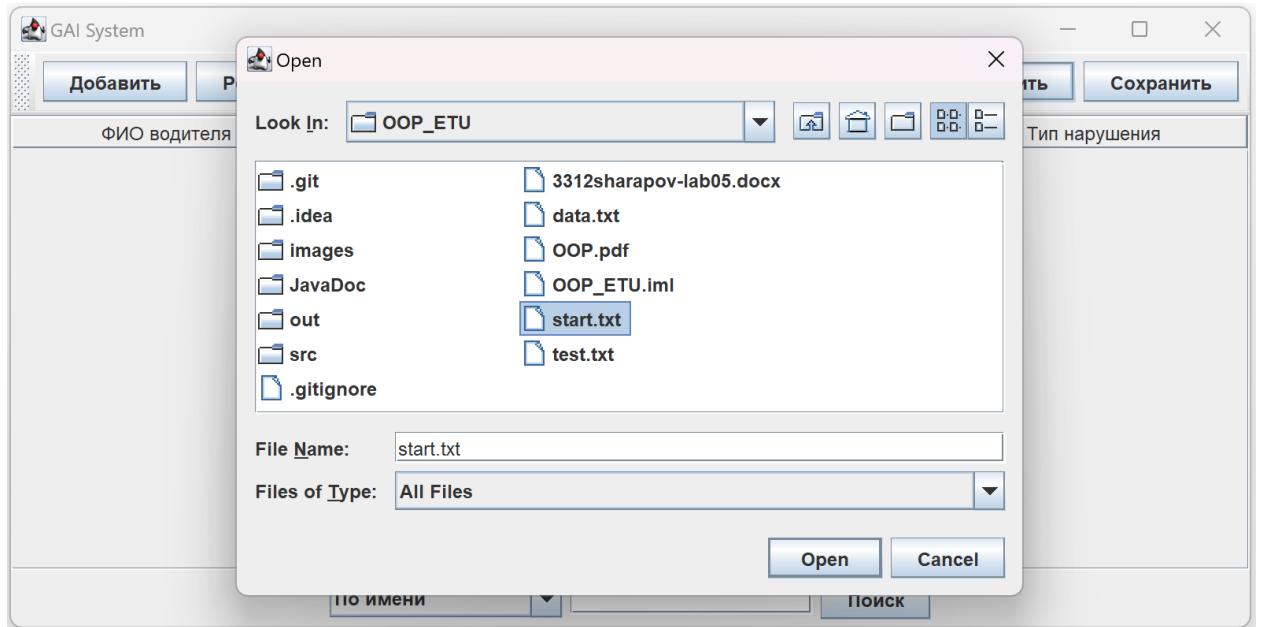


Рисунок 1 – Загрузка данных из файла

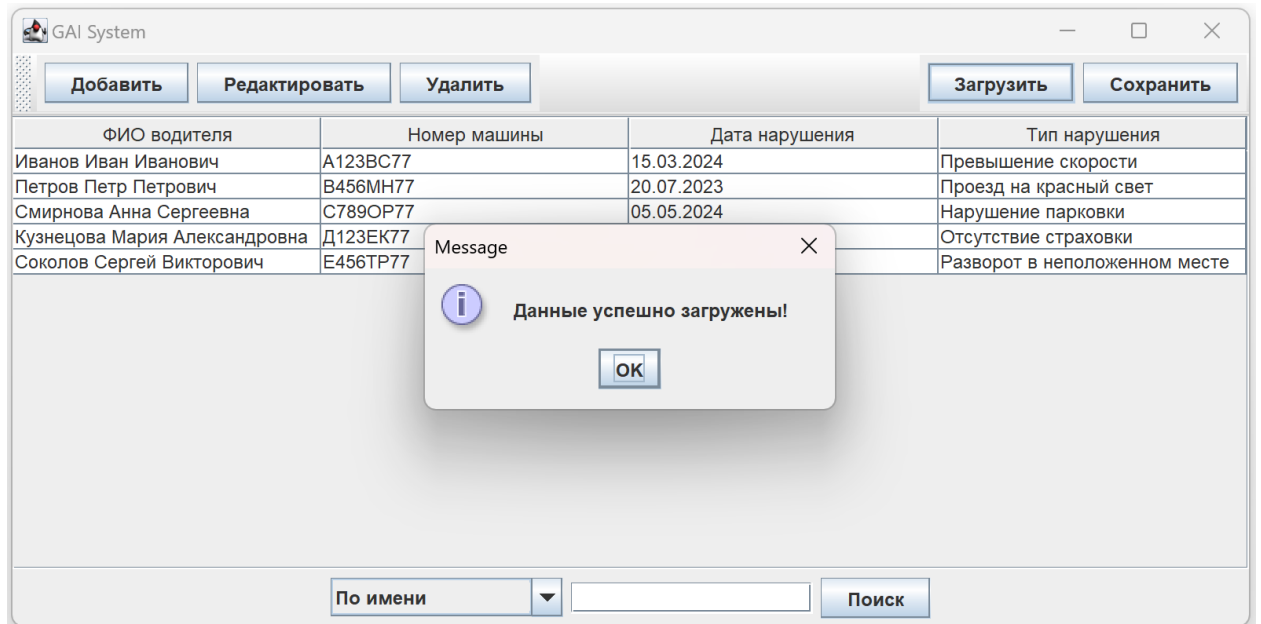


Рисунок 2 – Уведомление, что данные загружены

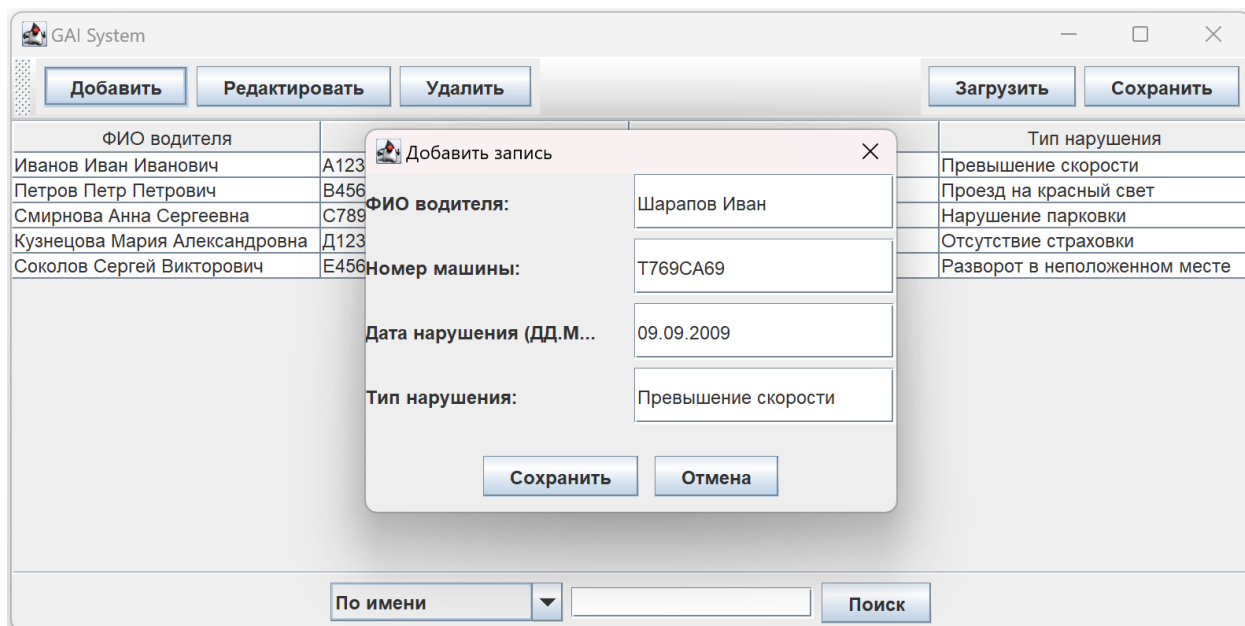


Рисунок 3 – Окно добавления (данные с ошибкой)

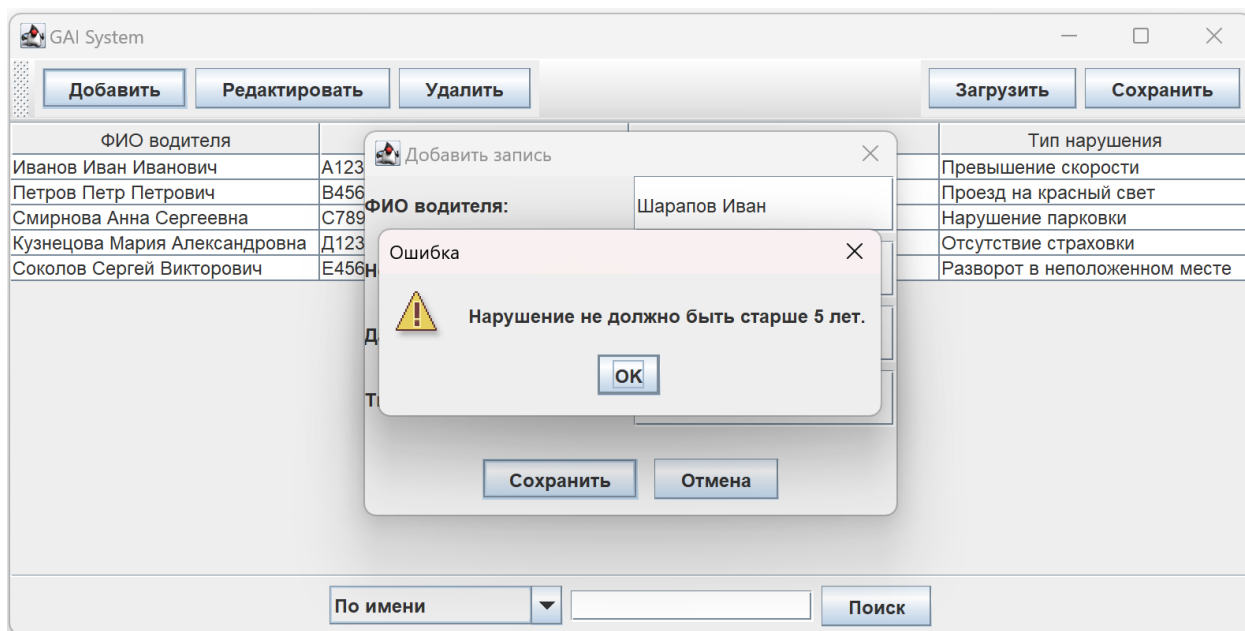


Рисунок 4 – Уведомление об ошибке в данных

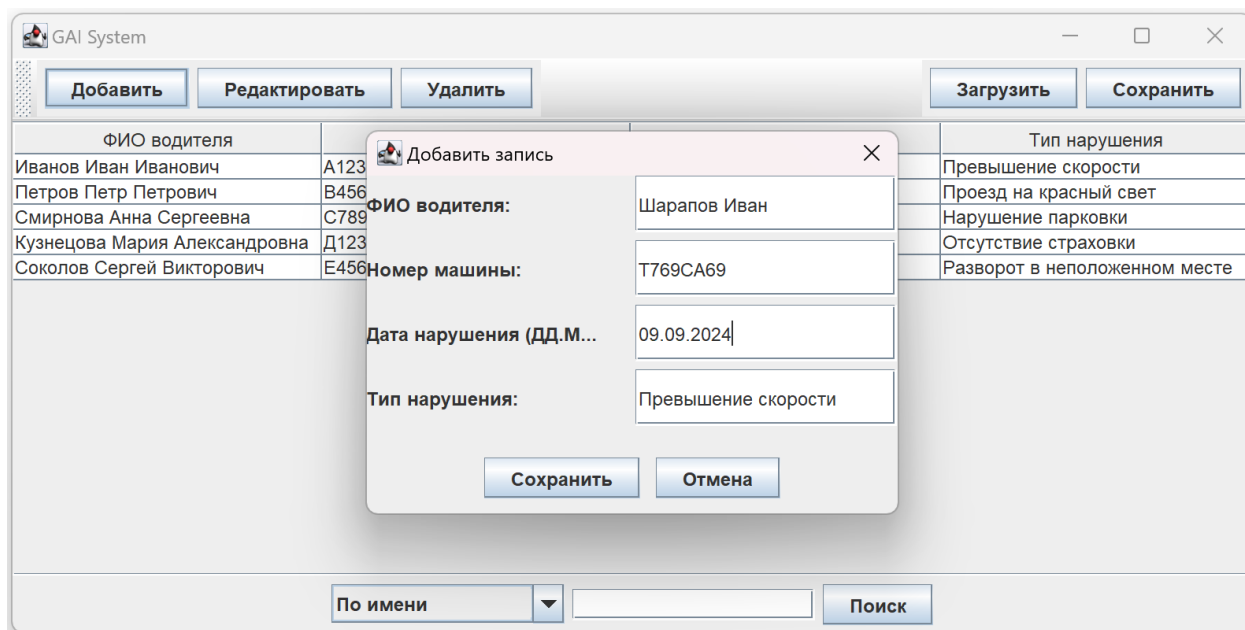


Рисунок 5 – Добавление данных (правильные данные)

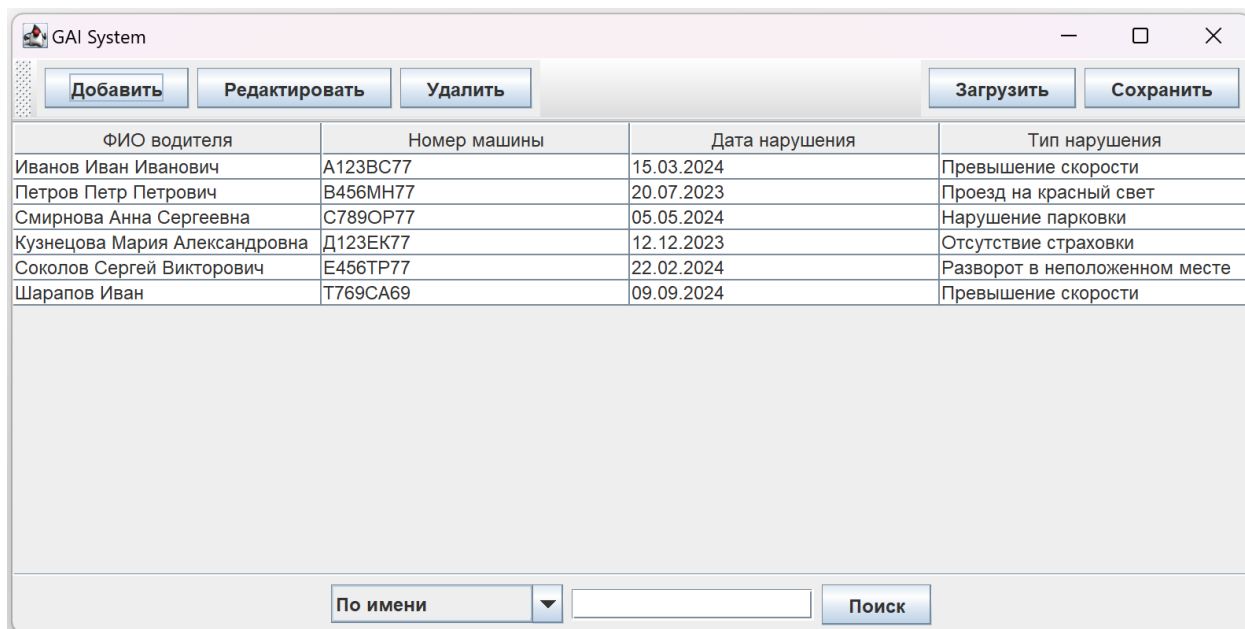


Рисунок 6 – Таблица после добавления

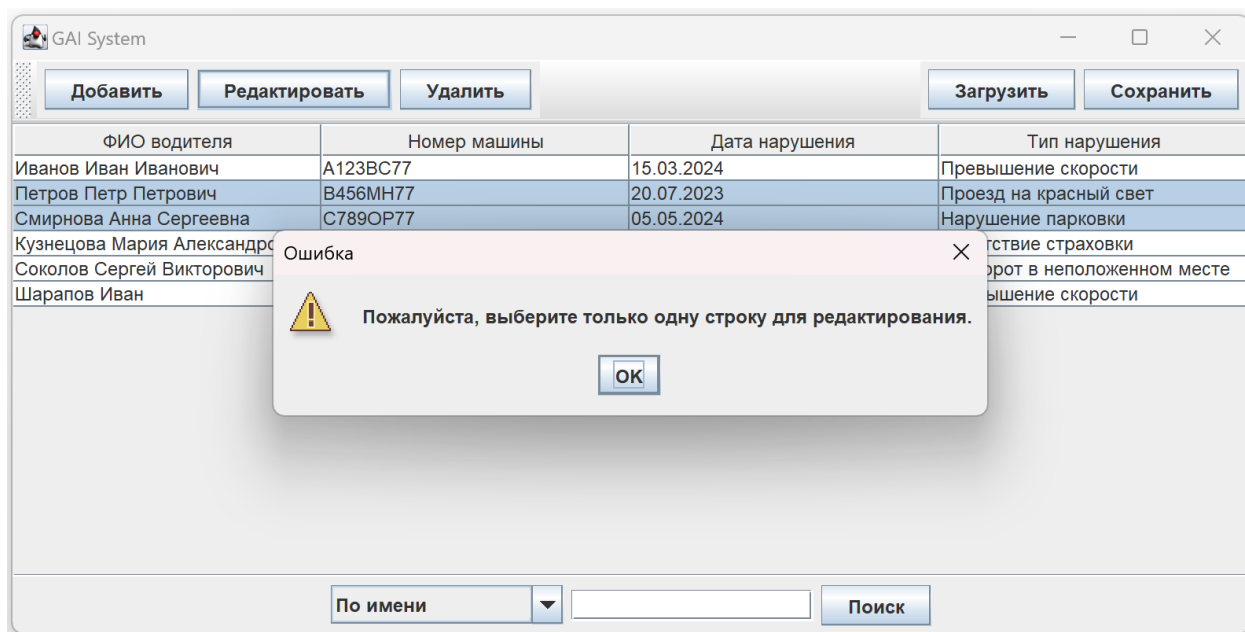


Рисунок 7 – Уведомление, что выбрано несколько строк для редактирования

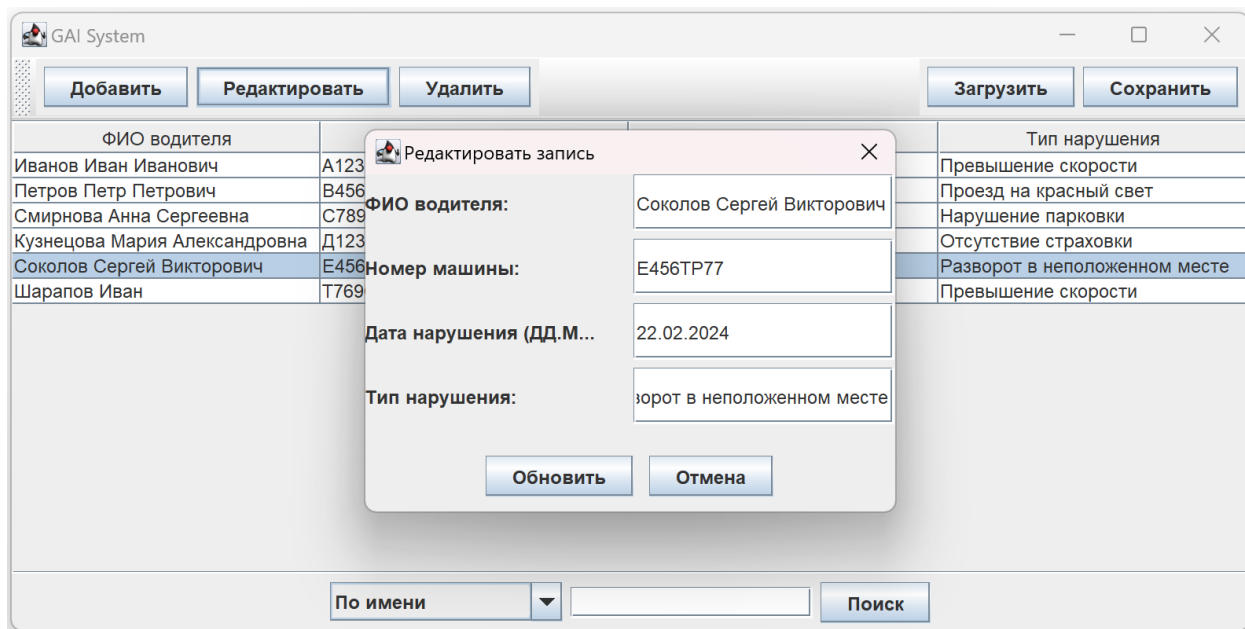


Рисунок 8 – Окно редактирования

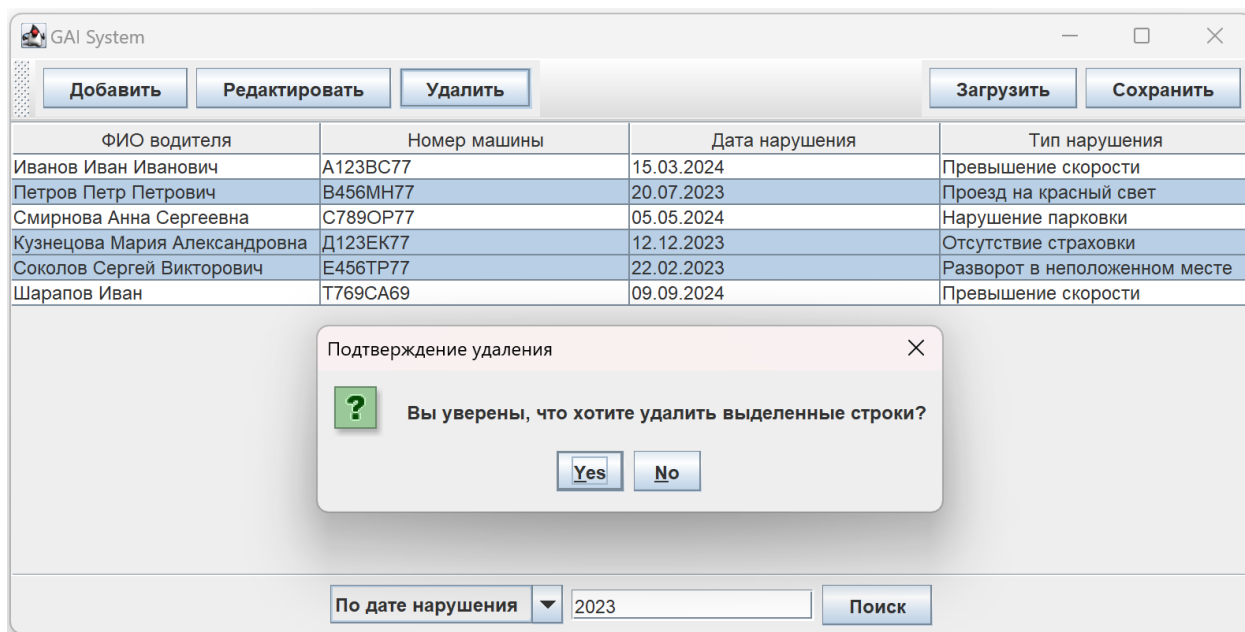


Рисунок 9 – Подтверждение удаления после того, как поиск выделил все нарушения за 2023 год

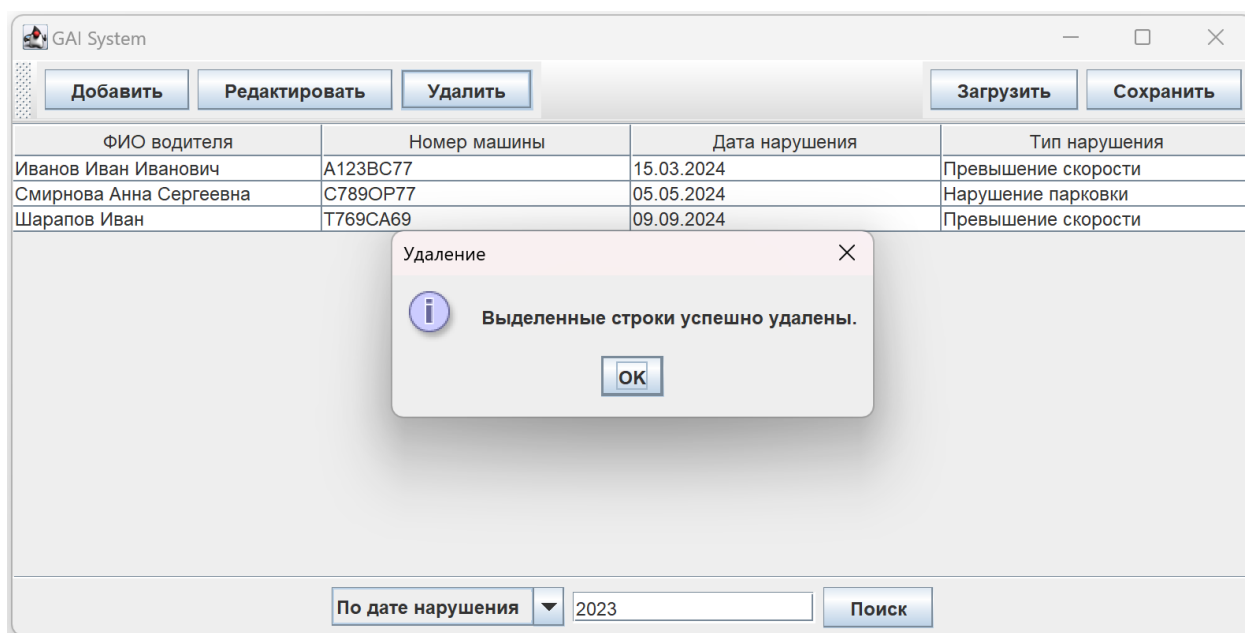


Рисунок 10 – Уведомление о успешном удалении

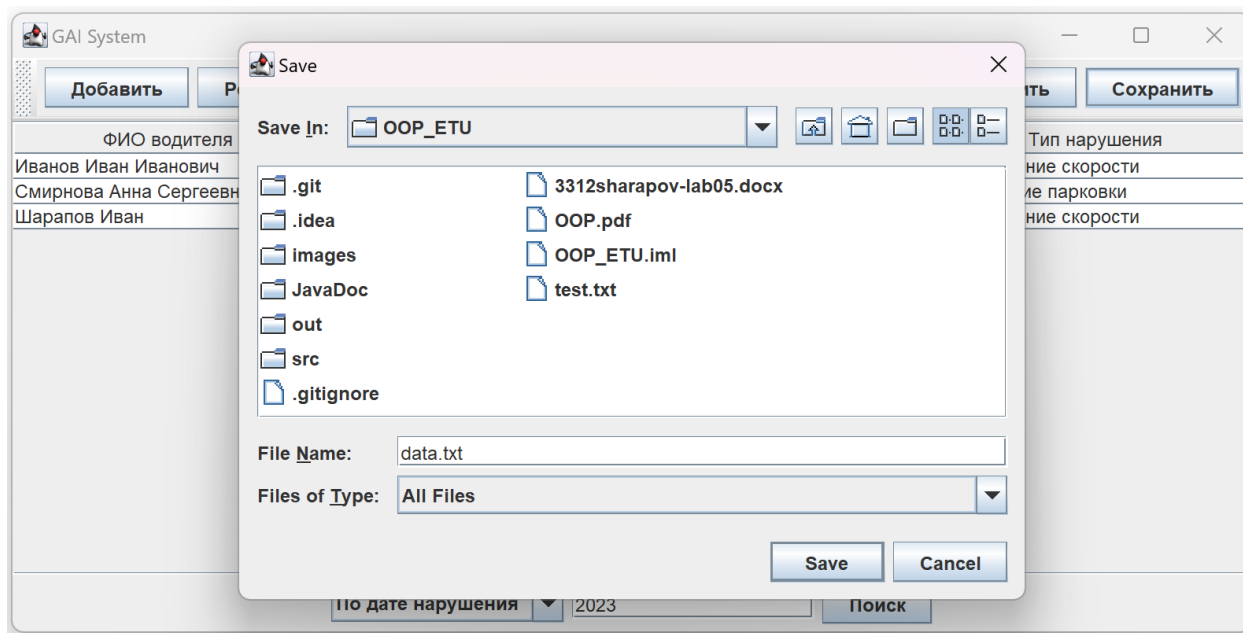


Рисунок 11 – Сохранение изменённых данных в файл

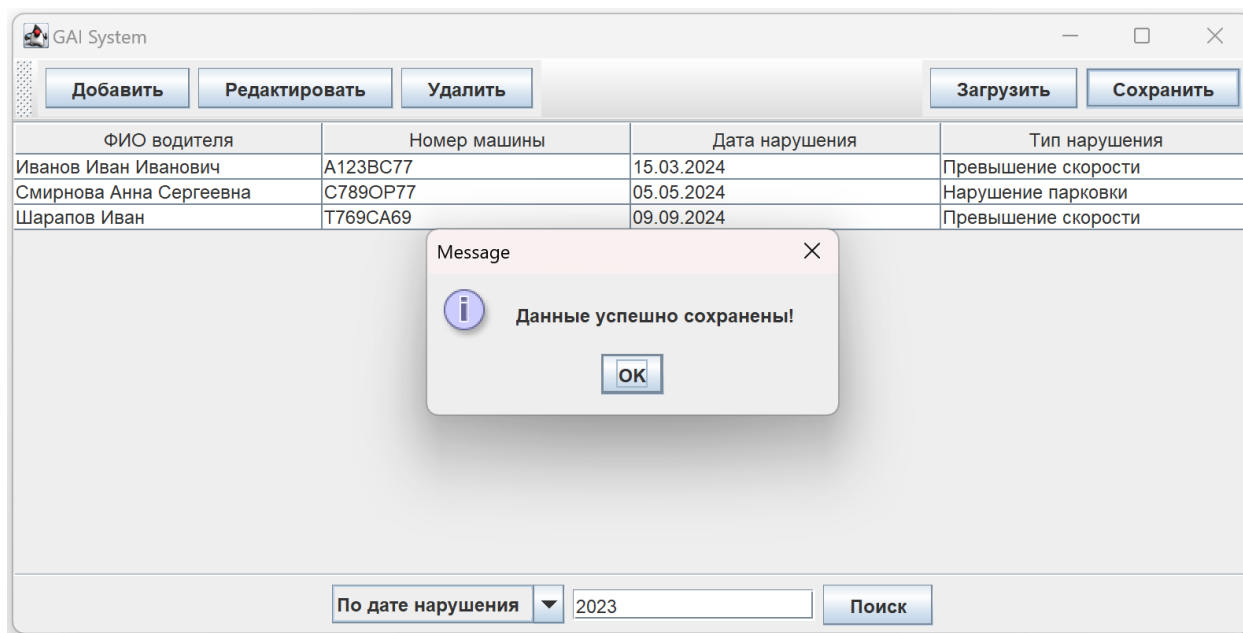


Рисунок 12 – Уведомление о успешном сохранении

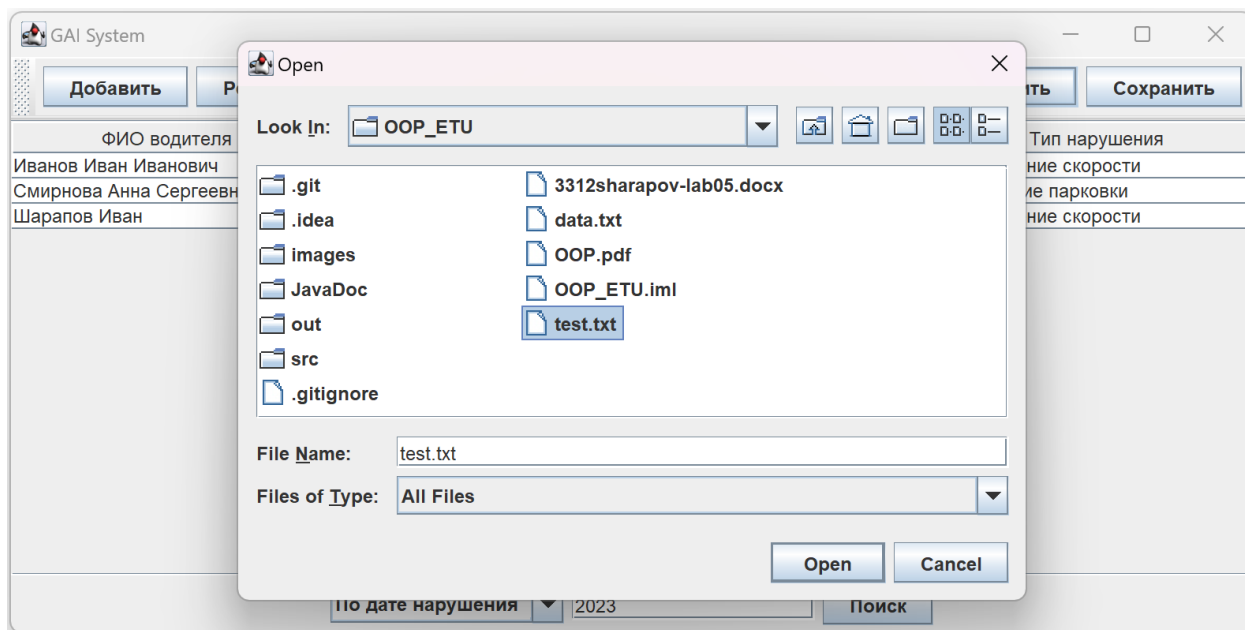


Рисунок 13 – Загрузка других данных

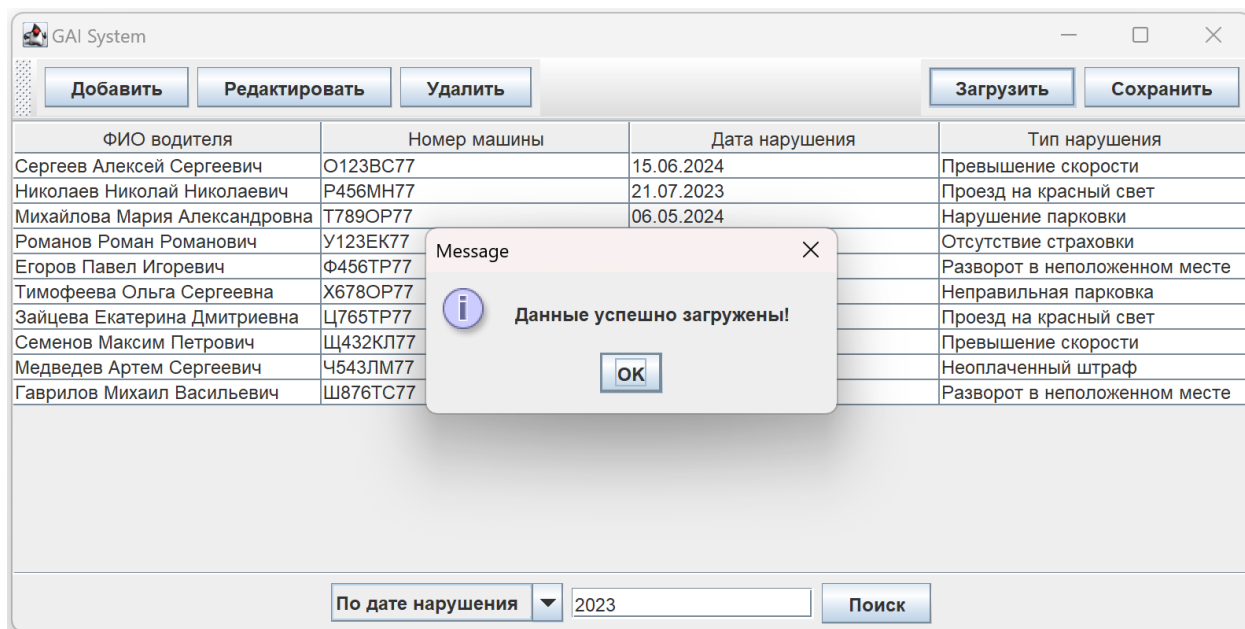


Рисунок 14 – Уведомление об успешной загрузке

Содержимое файлов

start.txt				
1	Иванов Иван Иванович	A123BC77	15.03.2024	Превышение скорости
2	Петров Петр Петрович	B456MN77	20.07.2023	Проезд на красный свет
3	Смирнова Анна Сергеевна	C789OP77	05.05.2024	Нарушение парковки
4	Кузнецова Мария Александровна	D123EK77	12.12.2023	Отсутствие страховки
5	Соколов Сергей Викторович	E456TP77	22.02.2024	Разворот в неположенном месте
6				

Рисунок 15 – Содержимое изначального файла

data.txt ×

1	Иванов Иван Иванович	A123BC77	15.03.2024	Превышение скорости
2	Смирнова Анна Сергеевна	C7890P77	05.05.2024	Нарушение парковки
3	Шарапов Иван	T769CA69	09.09.2024	Превышение скорости
4				

Рисунок 16 – Содержимое изменённых данных

test.txt ×

1	Сергеев Алексей Сергеевич	0123BC77	15.06.2024	Превышение скорости
2	Николаев Николай Николаевич	P456MN77	21.07.2023	Проезд на красный свет
3	Михайлова Мария Александровна	T7890P77	06.05.2024	Нарушение парковки
4	Романов Роман Романович	Y123EK77	13.12.2023	Отсутствие страховки
5	Егоров Павел Игоревич	Φ456TP77	23.02.2024	Разворот в неположенном месте
6	Тимофеева Ольга Сергеевна	X6780P77	11.08.2023	Неправильная парковка
7	Зайцева Екатерина Дмитриевна	Ц765TP77	06.09.2024	Проезд на красный свет
8	Семенов Максим Петрович	Щ432KL77	04.10.2024	Превышение скорости
9	Медведев Артем Сергеевич	Ч543LM77	15.02.2023	Неоплаченный штраф
10	Гаврилов Михаил Васильевич	Ш876TC77	23.05.2024	Разворот в неположенном месте
11				

Рисунок 17 – Содержимое дополнительных данных

Текст программы

Класс Main.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;

import Exceptions.*;

/**
 * Программа для работы с данными о водителях и их нарушениях.
 * Содержит функции добавления, редактирования, удаления записей, а также сохранения и
 * загрузки данных в файл.
 *
 * @author Шарапов Иван 3312
 * @version 1.3
 */
public class Main {
    private JFrame mainWindow;
    private DefaultTableModel tableModel;
    private JTable dataTable;
    private JButton addDriverButton, editDriverButton, deleteDriverButton, loadDriverButton,
    saveDriverButton;
    private JTextField searchField;
    private JComboBox<String> searchTypeComboBox;

    /**
     * Конструктор класса Main.
     * Инициализирует основное окно приложения для работы с данными.
     */
    public Main() {
        // Конструктор по умолчанию, который инициализирует класс Main.
    }
}
```

```

/**
 * Метод для создания и отображения главного окна программы.
 * Включает создание таблицы, панели инструментов с кнопками и панель поиска.
 */
public void show() {
    // Создание основного окна приложения
    mainFrame = new JFrame("GAI System");
    mainFrame.setSize(800, 400);
    mainFrame.setLocation(100, 100);
    mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Создание кнопок для управления записями
    addDriverButton = new JButton("Добавить");
    editDriverButton = new JButton("Редактировать");
    deleteDriverButton = new JButton("Удалить");
    loadDriverButton = new JButton("Загрузить");
    saveDriverButton = new JButton("Сохранить");

    // Панель инструментов, которая содержит кнопки
    JToolBar toolBar = new JToolBar("Toolbar");
    toolBar.setLayout(new BorderLayout()); // Устанавливаем BorderLayout для панели инструментов

    // Панель с кнопками управления (добавление, редактирование, удаление)
    JPanel leftPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
    leftPanel.add(addDriverButton);
    leftPanel.add(editDriverButton);
    leftPanel.add(deleteDriverButton);
    toolBar.add(leftPanel, BorderLayout.WEST); // Размещаем в левой части панели

    // Панель с кнопками сохранения и загрузки
    JPanel rightPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    rightPanel.add(loadDriverButton);
    rightPanel.add(saveDriverButton);
    toolBar.add(rightPanel, BorderLayout.EAST); // Размещаем в правой части панели

    mainFrame.setLayout(new BorderLayout());
    mainFrame.add(toolBar, BorderLayout.NORTH); // Размещение панели инструментов сверху

    // Создание таблицы для отображения данных
    String[] columns = {"ФИО водителя", "Номер машины", "Дата нарушения", "Тип нарушения"};
    String[][] data = {
        {"Иванов Иван Иванович", "A123BC77", "15.03.2024", "Превышение скорости"},
        {"Петров Петр Петрович", "B456MN77", "20.07.2023", "Проезд на красный свет"},
        {"Смирнова Анна Сергеевна", "C789OP77", "05.05.2024", "Нарушение парковки"},
        {"Кузнецова Мария Александровна", "D123EK77", "12.12.2023", "Отсутствие страховки"},
        {"Соколов Сергей Викторович", "E456TP77", "22.02.2024", "Разворот в неположенном месте"}
    };
    tableModel = new DefaultTableModel(data, columns);
    dataTable = new JTable(tableModel); // Таблица, которая использует данные tableModel
    JScrollPane scrollPane = new JScrollPane(dataTable); // Добавляем прокрутку для таблицы
    mainFrame.add(scrollPane, BorderLayout.CENTER); // Размещаем таблицу в центре окна

    // Элементы поиска: поле ввода и кнопка "Поиск"
    searchTypeComboBox = new JComboBox<>(new String[]{"По имени", "По номеру машины", "По дате нарушения", "По типу нарушения"});
    searchField = new JTextField(15);
    JButton searchButton = new JButton("Поиск");

    JPanel searchPanel = new JPanel();
    searchPanel.add(searchTypeComboBox);
    searchPanel.add(searchField);
    searchPanel.add(searchButton);
    mainFrame.add(searchPanel, BorderLayout.SOUTH); // Панель поиска размещается внизу

    // Добавляем действия для кнопок

    // "Добавить" — действие для добавления новой записи
    addDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            new RecordDialog(mainFrame, tableModel, -1).setVisible(true); // -1
        }
    });
}

```

означает, что это добавление новой записи

```

    });

    // "Редактировать" — действие для редактирования выбранной записи
    editDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            int[] selectedRows = dataTable.getSelectedRows();

            // Проверка, что выбрана ровно одна строка
            if (selectedRows.length != 1) {
                JOptionPane.showMessageDialog(mainFrame, "Пожалуйста, выберите только
одну строку для редактирования.", "Ошибка", JOptionPane.WARNING_MESSAGE);
                return;
            }

            // Открытие диалогового окна для редактирования выбранной строки
            int selectedRow = selectedRows[0];
            new RecordDialog(mainFrame, tableModel, selectedRow).setVisible(true);
        }
    });

    // "Удалить" — действие для удаления выбранной записи
    deleteDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            // Проверяем, есть ли выделенные строки
            int[] selectedRows = dataTable.getSelectedRows();
            if (selectedRows.length == 0) {
                JOptionPane.showMessageDialog(mainFrame, "Нет выделенных строк для
удаления.", "Ошибка", JOptionPane.WARNING_MESSAGE);
                return;
            }

            // Запрос подтверждения у пользователя
            int confirm = JOptionPane.showConfirmDialog(mainFrame, "Вы уверены, что
хотите удалить выделенные строки?", "Подтверждение удаления", JOptionPane.YES_NO_OPTION);
            if (confirm == JOptionPane.YES_OPTION) {
                // Удаляем строки с конца списка, чтобы избежать смещения индексов
                for (int i = selectedRows.length - 1; i >= 0; i--) {
                    tableModel.removeRow(selectedRows[i]);
                }
                JOptionPane.showMessageDialog(mainFrame, "Выделенные строки успешно
удалены.", "Удаление", JOptionPane.INFORMATION_MESSAGE);
            }
        }
    });

    // "Загрузить" — открывает диалоговое окно для выбора файла и загружает данные в
таблицу
    loadDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            JFileChooser fileChooser = new JFileChooser();
            fileChooser.setCurrentDirectory(new File(System.getProperty("user.dir")));
            // Устанавливаем корневую директорию проекта
            int returnValue = fileChooser.showOpenDialog(mainFrame); // Показываем
диалог открытия файла
            if (returnValue == JFileChooser.APPROVE_OPTION) { // Если файл выбран
                File selectedFile = fileChooser.getSelectedFile(); // Получаем выбранный
файл
                loadDataFromFile(selectedFile); // Загружаем данные из файла в таблицу
            }
        }
    });

    // "Сохранить" — открывает диалоговое окно для сохранения файла и записывает данные
таблицы в файл
    saveDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            JFileChooser fileChooser = new JFileChooser();
            fileChooser.setCurrentDirectory(new File(System.getProperty("user.dir")));
            // Устанавливаем корневую директорию проекта
            int returnValue = fileChooser.showSaveDialog(mainFrame); // Показываем
диалог сохранения файла
            if (returnValue == JFileChooser.APPROVE_OPTION) { // Если выбрано место для
сохранения

```

```

        File selectedFile = fileChooser.getSelectedFile(); // Получаем файл,
        который выбрал пользователь
        saveDataToFile(selectedFile); // Сохраняем данные таблицы в этот файл
    }
});

// "Поиск" — выполняет поиск в таблице, по введённой строке
searchButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        try {
            validateSearchField(searchField); // Проверка значений в поле поиска
            performSearch(searchField.getText()); // Выполнение поиска по
            введённому тексту
        } catch (NullPointerException ex) {
            JOptionPane.showMessageDialog(mainFrame, ex.getMessage(), "Ошибка",
            JOptionPane.ERROR_MESSAGE);
        } catch (EmptySearchException ex) {
            JOptionPane.showMessageDialog(mainFrame, ex.getMessage(), "Ошибка",
            JOptionPane.WARNING_MESSAGE);
        }
    }
});

mainFrame.setVisible(true); // Делаем главное окно видимым
}

/**
 * Загружает данные из указанного файла в таблицу.
 *
 * @param file файл, из которого будут загружены данные.
 */
private void loadDataFromFile(File file) {
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        tableModel.setRowCount(0); // Очищаем текущие данные таблицы перед загрузкой
        новых
        while ((line = reader.readLine()) != null) {
            // Разделяем строку по табуляциям, чтобы получить значения для каждой
            колонок
            String[] rowData = line.split("\t");
            tableModel.addRow(rowData); // Добавляем новую строку в таблицу
        }
        JOptionPane.showMessageDialog(mainFrame, "Данные успешно загружены!"); //
        Сообщение об успешной загрузке
    } catch (IOException e) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка при загрузке данных."); //
        Сообщение об ошибке
    }
}

/**
 * Сохраняет данные из таблицы в указанный файл.
 *
 * @param file файл, в который будут сохранены данные.
 */
private void saveDataToFile(File file) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(file))) {
        // Проходим по каждой строке таблицы
        for (int row = 0; row < tableModel.getRowCount(); row++) {
            for (int col = 0; col < tableModel.getColumnCount(); col++) {
                // Записываем значения ячеек таблицы в файл через табуляцию
                writer.write(tableModel.getValueAt(row, col).toString() + "\t");
            }
            writer.newLine(); // Переход на новую строку после каждой записи
        }
        JOptionPane.showMessageDialog(mainFrame, "Данные успешно сохранены!"); //
        Сообщение об успешном сохранении
    } catch (IOException e) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка при сохранении данных."); //
        Сообщение об ошибке
    }
}

/**
 * Возвращает индекс столбца для поиска на основе выбранного поля.
 */

```

```

    * @param selectedField Поле, выбранное пользователем в JComboBox для поиска
    * @return Индекс столбца для поиска, или -1, если поле не распознано
    */
    private int getColumnIndex(String selectedField) {
        switch (selectedField) {
            case "По имени":
                return 0; // Индекс столбца с ФИО водителя
            case "По номеру машины":
                return 1; // Индекс столбца с номером машины
            case "По дате нарушения":
                return 2; // Индекс столбца с датой нарушения
            case "По типу нарушения":
                return 3; // Индекс столбца с типом нарушения
            default:
                return -1;
        }
    }

    /**
     * Проверяет значение в поле поиска и генерирует исключения, если поле пустое или
     * содержит null.
     *
     * @param searchField Поле ввода текста для поиска
     * @throws EmptySearchException Если поле ввода пустое
     * @throws NullPointerException Если значение в поле null
     */
    private void validateSearchField(JTextField searchField) throws EmptySearchException,
        NullPointerException {
        String searchText = searchField.getText();
        if (searchText == null) {
            throw new NullPointerException("Поисковый запрос отсутствует");
        }
        if (searchText.isEmpty()) {
            throw new EmptySearchException();
        }
    }

    /**
     * Выполняет поиск по таблице и выделяет все строки, содержащие указанный текст в
     * выбранном поле, без учета регистра.
     *
     * @param query Строка для поиска
     */
    private void performSearch(String query) {
        dataTable.clearSelection(); // Снимаем предыдущее выделение
        boolean found = false;

        // Приводим запрос к нижнему регистру
        String lowerCaseQuery = query.toLowerCase();

        // Получаем индекс столбца для поиска
        int columnIndex = getColumnIndex((String) searchTypeComboBox.getSelectedItem());
        if (columnIndex == -1) {
            JOptionPane.showMessageDialog(mainFrame, "Некорректное поле для поиска",
                "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Проходим по всем строкам, но только в выбранном столбце
        for (int i = 0; i < tableModel.getRowCount(); i++) {
            String cellValue = tableModel.getValueAt(i,
                columnIndex).toString().toLowerCase();

            // Если значение ячейки содержит искомый текст без учета регистра
            if (cellValue.contains(lowerCaseQuery)) {
                dataTable.addRowSelectionInterval(i, i); // Выделяем строку
                found = true;
            }
        }

        if (!found) {
            JOptionPane.showMessageDialog(mainFrame, "Совпадения не найдены", "Результат
                поиска", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    /**
     * Точка входа в приложение. Запускает метод show() для отображения GUI.

```



```

    *
    * @param args Аргументы командной строки (не используются).
    */
    public static void main(String[] args) {
        new Main().show(); // Запуск приложения
    }
}

```

Класс RecordDialog.java

```

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

/**
 * Диалоговое окно для добавления или редактирования записи в таблице.
 */
public class RecordDialog extends JDialog {

    /**
     * Поле для ввода ФИО водителя.
     */
    private JTextField nameField;

    /**
     * Поле для ввода номера машины.
     */
    private JTextField licenseField;

    /**
     * Поле для ввода даты нарушения.
     */
    private JTextField dateField;

    /**
     * Поле для ввода типа нарушения.
     */
    private JTextField violationField;

    /**
     * Модель таблицы, используемая для добавления или обновления записи.
     */
    private DefaultTableModel tableModel;

    /**
     * Индекс строки для редактирования, -1 если добавляется новая запись.
     */
    private int rowIndex = -1;

    /**
     * Конструктор для создания окна записи (добавление или редактирование).
     *
     * @param parentFrame Родительское окно
     * @param tableModel Модель таблицы
     * @param rowIndex Индекс строки для редактирования (-1 для добавления новой записи)
     */
    public RecordDialog(JFrame parentFrame, DefaultTableModel tableModel, int rowIndex) {
        super(parentFrame, rowIndex == -1 ? "Добавить запись" : "Редактировать запись",
            true);
        this.tableModel = tableModel;
        this.rowIndex = rowIndex;

        setSize(350, 250);
        setLocationRelativeTo(parentFrame);
        setLayout(new BorderLayout(10, 10));

        // Панель с полями ввода
        JPanel inputPanel = new JPanel(new GridLayout(4, 2, 5, 5));
        nameField = new JTextField(15);
        licenseField = new JTextField(10);
        dateField = new JTextField(10);
        violationField = new JTextField(15);

        inputPanel.add(new JLabel("ФИО водителя:"));
        inputPanel.add(nameField);
        inputPanel.add(new JLabel("Номер машины:"));
        inputPanel.add(licenseField);
        inputPanel.add(new JLabel("Дата нарушения (ДД.ММ.ГГГГ):"));
        inputPanel.add(dateField);
        inputPanel.add(new JLabel("Тип нарушения:"));
        inputPanel.add(violationField);
    }
}

```

```

inputPanel.add(dateField);
inputPanel.add(new JLabel("Тип нарушения:"));
inputPanel.add(violationField);

// Заполнение полей, если это редактирование
if (rowIndex != -1) {
    nameField.setText((String) tableModel.getValueAt(rowIndex, 0));
    licenseField.setText((String) tableModel.getValueAt(rowIndex, 1));
    dateField.setText((String) tableModel.getValueAt(rowIndex, 2));
    violationField.setText((String) tableModel.getValueAt(rowIndex, 3));
}

// Панель с кнопками
JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));
JButton saveButton = new JButton(rowIndex == -1 ? "Сохранить" : "Обновить");
JButton cancelButton = new JButton("Отмена");

saveButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (validateAndSave()) {
            dispose();
        }
    }
});

cancelButton.addActionListener(e -> dispose());

buttonPanel.add(saveButton);
buttonPanel.add(cancelButton);

// Добавление панелей в диалоговое окно
add(inputPanel, BorderLayout.CENTER);
add(buttonPanel, BorderLayout.SOUTH);
}

/**
 * Проверяет корректность данных и сохраняет их, если данные верны.
 *
 * @return true, если данные успешно сохранены, иначе false
 */
private boolean validateAndSave() {
    if (nameField.getText().isEmpty() || licenseField.getText().isEmpty() ||
        dateField.getText().isEmpty() || violationField.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Все поля должны быть заполнены.", "Ошибка",
            JOptionPane.WARNING_MESSAGE);
        return false;
    }

    // Проверка формата российского номера
    if (!licenseField.getText().matches("^([АВЕКМНОРСТУХ]{1}\\d{3}[АВЕКМНОРСТУХ]{2}\\d{2,3}$")) {
        JOptionPane.showMessageDialog(this, "Неверный формат номера. Введите российский номер (например, А123BC77).", "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    }

    // Проверка даты нарушения
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd.MM.yyyy");
    LocalDate violationDate;
    try {
        violationDate = LocalDate.parse(dateField.getText(), formatter);
    } catch (DateTimeParseException ex) {
        JOptionPane.showMessageDialog(this, "Неверный формат даты. Используйте ДД.ММ.ИТТТ.", "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    }

    LocalDate currentDate = LocalDate.now();
    LocalDate fiveYearsAgo = currentDate.minusYears(5);

    // Проверка: дата не должна быть в будущем и не старше 5 лет
    if (violationDate.isAfter(currentDate)) {
        JOptionPane.showMessageDialog(this, "Дата нарушения не может быть в будущем.", "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    } else if (violationDate.isBefore(fiveYearsAgo)) {
        JOptionPane.showMessageDialog(this, "Нарушение не должно быть старше 5 лет.",

```



```

"Ошибка", JOptionPane.WARNING_MESSAGE);
    return false;
}

// Сохранение данных
if (rowIndex == -1) {
    // Добавление новой записи
    tableModel.addRow(new Object[]{nameField.getText(), licenseField.getText(),
dateField.getText(), violationField.getText()});
} else {
    // Обновление существующей записи
    tableModel.setValueAt(nameField.getText(), rowIndex, 0);
    tableModel.setValueAt(licenseField.getText(), rowIndex, 1);
    tableModel.setValueAt(dateField.getText(), rowIndex, 2);
    tableModel.setValueAt(violationField.getText(), rowIndex, 3);
}
return true;
}
}

```

Класс EmptySearchException.java

```

package Exceptions;

/**
 * Исключение, вызываемое, когда поле поиска пустое.
 */
public class EmptySearchException extends Exception {

    /**
     * Конструктор без параметров, создающий исключение с сообщением об ошибке пустого поля
     * поиска.
     */
    public EmptySearchException() {
        super("Поле поиска не должно быть пустым");
    }
}

```

Приложение

Ссылка на видео:

<https://rutube.ru/video/private/990c38e286c09fc127219b5ed55cf4b4/?p=zcxhgV-RzBeJhvFVb3iUVw>

Ссылка на репозиторий:

https://github.com/DexTver/OOP_ETU/tree/lab_05